

Arduino for Embedded Systems - Beginner Level

Lesson 7: Serial Communication and Data Logging

Serial Communication and Data Logging – Study Notes

Key Concepts

1. **UART/Serial Communication** – How microcontrollers exchange byte level data with a PC (TX, RX, baud rate, start/stop bits).
2. **Serial Monitor** – The IDE's built in terminal that displays incoming data and can send commands back to the board.
3. **Data Formatting for Logging** – Converting sensor readings into a delimited string (e.g., CSV) that a PC can easily parse.
4. **Timing & Buffer Management** – Ensuring the microcontroller sends data at a rate the PC can receive without overflow.
5. **Saving to CSV** – Capturing the Serial Monitor output (or using a script) to create a comma separated values file for later analysis.

Summary

Serial communication is the most straightforward way to move data from a microcontroller (Arduino, ESP32, etc.) to a computer. The board's UART peripheral sends bytes over a TX line, while the PC's USB to UART bridge receives them at a pre agreed baud rate (commonly 9600 /bps for beginners). In the Arduino IDE, the **Serial Monitor** visualizes this stream in real time, allowing you to verify that sensor values are being transmitted correctly.

To log data, each sensor reading is formatted as a line of text, typically using commas to separate fields (e.g., `timestamp,temperature,humidity`). When the Serial Monitor is open, you can copy paste the output into a text editor or, more conveniently, use the IDE's "Save as .txt" feature and then rename the file to `.csv`. The resulting CSV file can be opened in Excel, Google Sheets, or any data analysis tool for further processing.

Important Points

- **Baud Rate Must Match** – Both the microcontroller (`Serial.begin(9600);`) and the Serial Monitor must be set to the same speed.
- **Use `Serial.println()`** – Sends the data string followed by a newline, which creates a new row in the CSV file.
- **CSV Formatting** – Separate values with commas; avoid commas inside a field or escape them. Example: ``millis(),analogRead(A0),temperature``.
- **Buffer Limits** – The Arduino's serial buffer is ~64 /bytes. Keep each line short or add a small delay (`delay(10);`) to avoid overflow.
- **Timestamping** – `millis()` or an RTC provides a time reference, making the log useful for trend analysis.

- ****Saving Data**** – In the Serial Monitor: *File !' Save As...* !' rename to `.csv`.
- ****Alternative: Python Script**** – A simple Python program can read the serial port continuously and write each line directly to a CSV file, automating the process.

Examples

Example 1 – Logging a Temperature Sensor to CSV

```
```cpp
// Arduino sketch
const int sensorPin = A0; // LM35 or similar
unsigned long lastPrint = 0;
const unsigned long interval = 1000; // 1 /s
void setup() {
 Serial.begin(9600);
 Serial.println("time_ms,temperature_C"); // CSV header
}
void loop() {
 if (millis() - lastPrint >= interval) {
 lastPrint = millis();
 int raw = analogRead(sensorPin);
 float voltage = raw * (5.0 / 1023.0);
 float temperature = voltage * 100.0; // LM35: 10 /mV/°C
 Serial.print(millis());
 Serial.print(',');
 Serial.println(temperature, 2); // two decimal places
 }
}
```
```

Explanation: The sketch prints a header line, then every second sends `time\_ms,temperature\_C`. When you open the Serial Monitor and ****Save As...****, you obtain a CSV file ready for Excel.

Example 2 – Python Logger for Multiple Sensors

```
```python
```

#### **python\_logger.py**

```
import serial
import csv
import time
ser = serial.Serial('COM3', 9600, timeout=1) # adjust port name
csv_file = open('sensor_log.csv', 'w', newline='')
writer = csv.writer(csv_file)
```

## Write header (optional, can be sent from Arduino)

```
writer.writerow(['timestamp_ms', 'temp_C', 'humidity_%'])
```

```
try:
```

```
 while True:
```

```
 line = ser.readline().decode('utf-8').strip()
```

```
 if line:
```

```
 # Arduino sends: "12345,23.5,45.2"
```

```
 fields = line.split(',')
```

```
 writer.writerow(fields)
```

```
 print(line) # echo to console
```

```
except KeyboardInterrupt:
```

```
 print("\nLogging stopped.")
```

```
finally:
```

```
 ser.close()
```

```
 csv_file.close()
```

```
...
```

**\*Explanation\*:** The script opens the serial port, reads each newline terminated line, splits it by commas, and appends it to `sensor\_log.csv`. This runs continuously until you press **\*\*Ctrl C\*\***, giving you a live log without manual copying.

---

## Quick Review

1. **\*\*What does `Serial.begin(9600);` do, and why must the Serial Monitor be set to the same value?\***
2. **\*\*Why is `Serial.println()` preferred over `Serial.print()` when creating a CSV log?\***
3. **\*\*Name two ways to save the incoming serial data as a CSV file.\*\*\***
4. **\*\*What could happen if you send data faster than the Serial Monitor can read it?\***
5. **\*\*How can you add a timestamp to each logged line without an external Real Time Clock?\***

---

## Further Reading

- **\*\*UART Protocol Details\*\*** – start/stop bits, parity, and common pitfalls.
- **\*\*Using an RTC (Real Time Clock) Module\*\*** for human readable timestamps.
- **\*\*Data Visualization\*\*** – importing CSV into Python (pandas, matplotlib) or Excel for plotting.
- **\*\*Advanced Serial Tools\*\*** – `screen`, `minicom`, `PuTTY`, and the `Serial Plotter` in Arduino IDE.
- **\*\*Wireless Logging\*\*** – BLE, Wi Fi, or LoRa alternatives to wired serial communication.

